

Advanced Algorithms: Homework 6 Solutions

Due on Mar. 15, 2024 at 11:59pm EST

Professor Dana Randall Spring 2024

As stated in the syllabus, unauthorized use
of previous semester course materials is
strictly prohibited in this course.

Exercise 1

Consider an online convex hull problem in which we are given the set s of n points one at a time. After receiving each point, we are to compute the convex hull of the points seen so far. Obviously we could run Graham's scan once for each point, with a total running time of $O(n^2 \log n)$. Show how to solve the online convex hull problem in a total of $O(n^2)$ time.

Solution

We recall that we can find the convex hull of points that are already in sorted order by x -coordinate in linear time. Also in linear time, if we have the vertices of a convex hull, we can put them in order of increasing x -coordinate in linear time by first finding the vertex with minimum x -coordinate, and then walking along the lower and upper hulls simultaneously, listing the vertices we see in order of their x -coordinate. Now note that any vertex on the convex hull of $p_1, p_2, \dots, p_i, p_{i+1}$ is either on the convex hull of $p_1, p_2, \dots, p_i$ or is point p_{i+1} . So, at each iteration, our online algorithm will put the convex hull vertices of $p_1, p_2, \dots, p_i$ in sorted order by their x -coordinate, insert p_{i+1} into its correct position in this sorted list, and then find the convex hull of these points. All this will happen in $O(n)$ time. Doing this for each of the n points gives a running time of $O(n^2)$, as desired.

Exercise 2

Given n points in the plane, construct a simple (i.e., non-self-intersecting) polygon having these points as its vertices. Devise an $O(n \log n)$ algorithm and show that this is optimal.

Solution

Compute the upper hull of all the points in $O(n \log n)$ time - as part of the algorithm, you've sorted all the points by x -coordinate. Take all the other points, as well as the left and right most points of the upper hull. For these points, connect them in order of increasing x -coordinate, going from top to bottom in the event of a tie. Now we have two paths, an upper path (the upper hull) and a lower path, connecting the leftmost and rightmost points. The segments forming the upper hull do not intersect. The new segments we add do not intersect either by construction. Finally, the segments we add do not intersect with the segments of the upper hull, by definition of the upper hull/convex hull. Thus this is a non self-intersecting polygon.

Now, suppose you can do this in less than $O(n \log n)$ time. I'll show this would imply you could also sort in less than $O(n \log n)$ time, a contradiction. Let $x_1, x_2, \dots, x_n$ be some numbers that we would like to sort. Make them into points in the plane by considering $(x_1, x_1^2), (x_2, x_2^2), \dots, (x_n, x_n^2)$. As we discussed in class, the convex hull of these points is all of the points, in order of x -value. Because the points are in convex position, this is also the only polygon with these points as vertices - if two points that aren't adjacent around the convex hull are connected by a polygon edge, this edge would divide the points in two and you wouldn't be able to connect points on opposite sides of the edge without having intersecting edges. If there was an algorithm for solving this problem in less than $O(n \log n)$ time, it would necessarily return the convex hull of these points, and you would be able to read off the numbers $x_1, x_2, \dots, x_n$ in sorted order. But then you've sorted the numbers in less than $O(n \log n)$ time, a contradiction.

Alternately, you could show $O(n \log n)$ is optimal by showing that once you find a polygon with the points as vertices, you could then use this to find the convex hull in linear time. Because finding the convex hull is necessarily $O(n \log n)$, then finding a polygon with these points as vertices must be as well. This requires proving some extra facts about polygons and convex hulls, though, for example that the points on the convex hull appear in order around the boundary of any polygon your algorithm finds.

Exercise 3

Given two arbitrary convex polygons P and Q with n vertices each, compute the convex hull of $P \cup Q$. Note that the two polygons can intersect any number of times, they can be disjoint, or one can be contained within the other.

- Specify as efficient an algorithm as you can to solve this problem, and analyze its complexity and prove its correctness.
- Use this algorithm to generate a divide-and-conquer algorithm without presorting that finds the convex hull of an arbitrary set of n points.

Solution

- Recall that we had a divide and conquer type algorithm for upper and lower hull. Separate our polygons P and Q into their upper and lower hulls in $O(n)$ time: find the points on P and Q with minimum and maximum x -coordinate each, and these separate the vertices of P and Q into two chains each. Here n is the total number of vertices in P and Q .

For now, consider the upper hull. We note that any vertex on the upper hull of $P \cup Q$ must be on the upper hull of P or on the upper hull of Q . In the decomposition, we get the points in the upper hulls of P and Q sorted by x -coordinate. We can merge these sorted lists for P and Q in linear time, and any vertices on the upper hull of $P \cup Q$ must be contained in this merged sorted list. Finally, use the upper hull merge algorithm done in class, which runs in $O(n)$ time when you have points that are already sorted by x -coordinate. This gives us a final merge in $O(n)$ time.

- Now, we can use divide and conquer. We first split the input points arbitrarily - we don't need to presort at all, we can just put half the points in one set and half in the other. We then find the convex hull of each subdivision recursively. Because we split our input points arbitrarily, these convex hull will probably not be disjoint, but that won't be a problem. Given the convex hulls of both subdivisions, we can use the algorithms of part (a) to merge them together, because this algorithm works even for polygons that overlap.

The recurrence is $T(n) = 2T(n/2) + O(n)$ which leads to an $O(n \log n)$ running time. Heuristically, we might expect this algorithm to be fairly fast because we don't presort the points.

As a heuristic, you can first check if the new point is inside the convex hull of all the previous points, and if it is do nothing, but this isn't necessary to get the desired $O(n^2)$ running time.

Exercise 4

Computational biologists often compute physical properties of large molecules. One of these properties is something called the *accessible surface area* of the molecule. We will consider this problem in a 2-dimensional setting.

We model the atoms of our molecule as a set of unit disks, each of radius of 1, centered at a given set of points $P = \{p_1, \dots, p_n\}$. Let b_i denote the unit disk centered at point p_i . Let $M(P) = \bigcup_{i=1}^n b_i$ denote the union of these disks (shaded in blue in Fig. 1(a)). Let $\partial M(P)$ denote its boundary. Observe that $\partial M(P)$ is composed of circular arcs, and it may have multiple connected components (all of which contribute to the accessible surface).

- Present an $O(n \log n)$ -time algorithm, which given a set P of n points in the plane, computes the length of accessible surface (see the solid curve in Fig. 1(a)).

Note: I am mostly interested in how to identify the circular arcs that make up the boundary of $M(P)$, not the actual perimeter value. To simplify matters, you may assume that you have access to a black-box function that computes the length of a given circular arc.

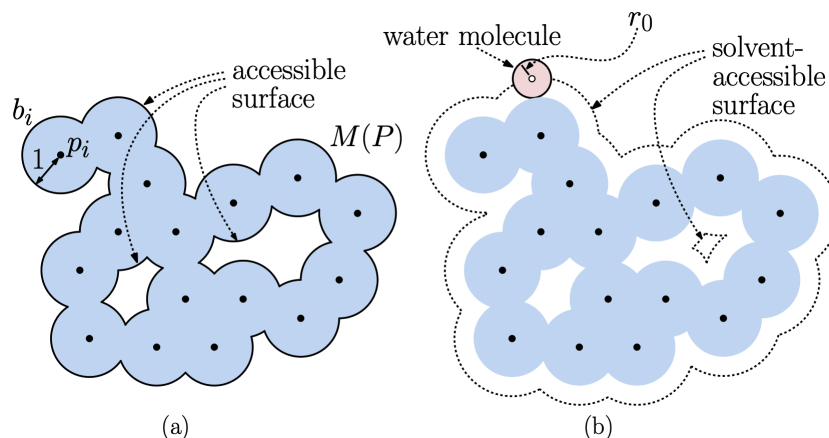


Figure 1: Solvent surface

- (b) In most applications in drug design, the molecule resides in liquid solvent, such as water. A statistic that is more relevant is something called the *solvent-accessible surface*. We model a water molecule as a disk of some radius r_0 . Imagine that we roll such a disk around the entire boundary of $M(P)$ and trace out the path taken by the center of this disk (the dashed curve in Fig. 1(b)). As with the accessible surface, this boundary may have multiple components, all of which contribute to the final result.

Present an $O(n \log n)$ -time algorithm, which given P and r_0 , computes the length of the solvent-accessible surface (the dashed curve in Fig. 1(b)). *Hint:* This should be an easy extension to (a).

Solution

- (a) **Design.** For each point p_i , let ∂b_i be the boundary of the unit ball centered at p_i . Now create the Voronoi diagram for P . For each point p_i , this will create a new Voronoi cell c_i :

$$c_i = \{p : \forall j, \|p - p_i\| \leq \|p - p_j\|\}.$$

The length of accessible surface is then given by

$$|\partial M(P)| = \sum_{i=1}^n |c_i \cap \partial b_i|.$$

Analysis. It suffices to prove

$$\partial M(P) = \bigcup_{i=1}^n (c_i \cap \partial b_i).$$

First, consider the formal definition for the two sets $\partial M(P)$ and $\bigcup_{i=1}^n (c_i \cap \partial b_i)$:

$$\partial M(P) = \{p : \exists i, \forall j, (\underbrace{\|p - p_i\| = 1}_{\text{on boundary}} \wedge \underbrace{\|p - p_j\| \geq 1}_{\text{not in interior}})\}$$

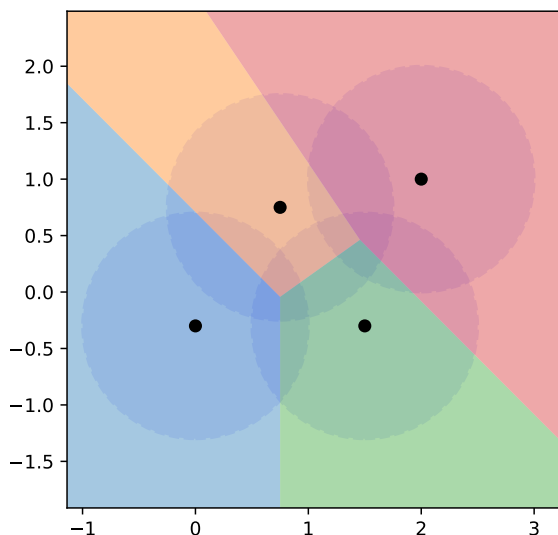
$$\bigcup_{i=1}^n (c_i \cap \partial b_i) = \{p : \exists i, \forall j, (\|p - p_i\| = 1 \wedge \|p - p_j\| \geq \|p - p_i\|)\}.$$

Notice the similarity between these definitions. In fact, if $x \in \partial M(P)$, then $\exists i, \forall j, \|p - p_i\| = 1$ and $\|p - p_j\| \geq 1 = \|p - p_i\|$, so $x \in \bigcup_{i=1}^n (c_i \cap \partial b_i)$.

Conversely, if $x \in \bigcup_{i=1}^n (c_i \cap \partial b_i)$, then $\|p - p_i\| = 1$ and $\|p - p_j\| \geq \|p - p_i\| \geq 1$, so $x \in \partial M(P)$. Thus the two sets are equal, and we are done.

I have also plotted a few molecules and the corresponding Voronoi diagram below using the code in the [Appendix](#); feel free to use this as a sanity check for any of the claims above.

Runtime. Computing the Voronoi diagram takes time $\mathcal{O}(n \log n)$. Computing the intersections of all Voronoi cells with their corresponding circles takes $\mathcal{O}(n)$ total time¹. Thus this algorithm takes time $\mathcal{O}(n \log n)$.



- (b) Simply run the algorithm in part (a) but change the radius of the unit ball to be $1 + r_0$. This correctly yields the length of the solvent-accessible surface because each point on the boundary of the original unit balls is displaced outward by a distance of r_0 , effectively increasing the radius of each circle by r_0 .

¹The Voronoi diagram has total complexity $\mathcal{O}(n)$, so the amortized number of halfspaces/sides per region is constant. For each side, we check whether it intersects the corresponding circle.

Appendix

Voronoi Diagram Code

```

import matplotlib.pyplot as plt
import numpy as np
from scipy.spatial import Voronoi

def voronoi_finite_polygons_2d(vor, radius=None):
    """
    Reconstruct infinite Voronoi regions in a 2D diagram to finite regions.
    """
    if vor.points.shape[1] != 2:
        raise ValueError("Requires 2D input")

    new_regions = []
    new_vertices = vor.vertices.tolist()

    center = vor.points.mean(axis=0)
    if radius is None:
        radius = vor.points.ptp().max()

    # Construct a map containing all ridges for a given point
    all_ridges = {}
    for (p1, p2), (v1, v2) in zip(vor.ridge_points, vor.ridge_vertices):
        all_ridges.setdefault(p1, []).append((p2, v1, v2))
        all_ridges.setdefault(p2, []).append((p1, v1, v2))

    # Reconstruct infinite regions
    for p1, region in enumerate(vor.point_region):
        vertices = vor.regions[region]

        if all(v >= 0 for v in vertices):
            # Finite region
            new_regions.append(vertices)
            continue

        # Reconstruct a non-finite region
        ridges = all_ridges[p1]
        new_region = [v for v in vertices if v >= 0]

        for p2, v1, v2 in ridges:
            if v2 < 0:
                v1, v2 = v2, v1
            if v1 >= 0:
                # Finite ridge: already in the region
                continue

            # Compute the missing endpoint of an infinite ridge
            t = vor.points[p2] - vor.points[p1] # Tangent
            t /= np.linalg.norm(t)
            n = np.array([-t[1], t[0]]) # Normal

```

```

        midpoint = vor.points[[p1, p2]].mean(axis=0)
        direction = np.sign(np.dot(midpoint - center, n)) * n
        far_point = vor.vertices[v2] + direction * radius

        new_region.append(len(new_vertices))
        new_vertices.append(far_point.tolist())

    # Sort region counterclockwise
    vs = np.asarray([new_vertices[v] for v in new_region])
    c = vs.mean(axis=0)
    angles = np.arctan2(vs[:, 1] - c[1], vs[:, 0] - c[0])
    new_region = np.array(new_region)[np.argsort(angles)]

    # Finish the region
    new_regions.append(new_region.tolist())

return new_regions, np.asarray(new_vertices)

points = np.array([[0, -0.3], [0.75, 0.75], [1.5, -0.3], [2, 1]])
vor = Voronoi(points)
regions, vertices = voronoi_finite_polygons_2d(vor, 1000) # Large radius to extend far

fig, ax = plt.subplots()

# Plot and slightly shade circles around the points
for point in points:
    circle = plt.Circle((point[0], point[1]), 1, color='blue', \
        fill=True, alpha=0.1, linestyle='--', linewidth=1.5)
    ax.add_artist(circle)

# Set equal aspect ratio
ax.set_aspect('equal', 'box')

# find the average of the points and then set the limits of the plot
center = points.mean(axis=0)
ax.set_xlim(center[0] - 2.2, center[0] + 2.2)
ax.set_ylim(center[1] - 2.2, center[1] + 2.2)

# Plot Voronoi
for region in regions:
    polygon = vertices[region]
    ax.fill(*zip(*polygon), alpha=0.4)

ax.plot(points[:, 0], points[:, 1], 'ko')

plt.savefig('voronoi.pdf', bbox_inches='tight')
```